

Active InferAnt Stream 014.2 ~ Generalized Notation

Notation for Generative Model Supply Chains

Source: [YouTube](#) **Playlist:** Active InferAnt Stream **Duration:** 1:57:07 |

Uploaded: Unknown **Transcript method:** auto_caption

All right. Hello, welcome everyone. I'm going to start this stream with a GitHub push. All right, it is June 8th, 2025 and this is active inferant stream 14.2. First, where we're going today in terms of AI art to quickly give a little visual beginning. No, I'll just show. Sorry about that. Was just working up to this last moment to get this going. Where we're headed is using RX infer multi-agent trajectory planning example and using GNN to give an exact reconfiguration. So we're going to be using generative not generalized notation notation to yield this simulation in RX infer. So there's a couple steps to go over. Uh this is just sort of a live working uh presentations and to be refined and polished. I just wanted to get it all into this one stream given all these recent updates. So to the stream agenda and also people watching live I will look forward to your comments and and ideas. Let's start with uh looking at the areas we'll go into. First just some overview on the tools I'm using cursor 1.0 which came out recently. GNN generalized notation notation. Check out stream 14.1 if you want to see more using LLM via cursor mainly claude 4 and it's going to be about reverse and forward engineering in the RX infer setting. Okay, where we're headed is a robust information supply chain from plain text configuration to executable simulation. So here's what that is going to look like again with the AI art. We have GNN the plain text GNN moving through all of these stages the triple play graphical on through executed simulation. So whether we think about that as meeting in the middle between the forward specification of GNN and all of these different kinds of things that we might want to do with GNN like the type checking all the modules of the package and the the golden spike moment is meeting in the middle with the reverse engineering of RX and fur examples and the forward engineering of the GNN package so that we can end up with this being in the middle between the plain text which gives us all these benefits of plain text reproducibility etc and all these functionalities of GNN. Uh in the main first section, I'll go over what that specifically was. Talk about the reverse engineering in the RX infer example from script to more of a package and then meeting that package with a

forward engineering pass from the GNN schema and pipeline on through the needed configuration. And that's the golden spike. Again, just a metaphor. Uh there have been many changes to the GNN package and still a few more that could occur with the logging and so on. But I'll go through a few sections of change and documentation little bit more on the golden spike and the analogy. Then we'll see what people comment where that goes. And I have a little bit of manual writing as well in terms of thinking about what that means from a organizational perspective and for active inference and for the institute and all that. Okay. Going to the package. All right. In the doc RX infer folder, the first thing is this short script clone RX infer examples. And this clones a repo which is a fork of the main RX infer examples that I have on my personal GitHub account. And this I sync it up with the main examples repo which is awesome that they're putting out more and more and different. And then in the support folder on my fork first there's a setup Julia script to get your environment ready. And then there's a notebooks to scripts Julia which takes the notebooks from the examples and it turns them all into scripts just by extracting out the code blocks. So they don't all necessarily work um but on the first pass but it gets all the content again to this forward reverse engineering idea. Here's the process. So first and the links to all this are in the video description. First use that notebooks to scripts script to uh convert the notebook into a script. Sometimes it might run away. Other times you might need to change certain things about the control flow. And first step is to just lock in the single script. So what that looks like in the case here in so then it clones in. This is work I did on the fork to get this is the the reverse engineering part with RX infer scripts advanced examples multi-agent trajectory archive. So this is the working single script version of the multi- agent model. Just for context, where this takes us is these four agents that do a negotiation of avoidance collision and obstacle uh avoidance. So different different collision avoidance and that's happening with at the heart of it all this app model block as always in RX infer that is used by the path planner inference step. This is the uh kind of the kernel and then RX infer is making that kernel of the generative model. The explicit probabilistic joint distribution makes it look a lot more like the math or the analytical description of the generative model and everything else is pushed into the inference methods. So that's that kind of idea of writing shorter app model blocks that distinguish different generative models and then everything else is like rendering that out into the graphical model with the stereotyped inference routines from that single script take it to more of a configural modular approach. So that was probably several hours, several dozen prompts, but not something that couldn't be done eventually in an automated fashion to split up that

single script that which was only lightly modified from the notebook format which is 339 lines to modify it from single script into in this case almost 10 different scripts. Some of them are really short, but they clarify and there's other ways that it probably could have been disassembled or or disarticulated. Um, this separates out different functions and also adds a lot of visualization and and logging methods. Third step from that configure uh from that multi-script package all the hardcoded variables take it out to a config toml file. So this is like a little domain specific language just for this one example in this one case where this gives all the configuration needed. So instead of just having the values coded in the scripts look to this central config standalone file there's multiple ways or stages of generalizing the variables like the dimensional state spaces initial parameterizations constraint variables number of agents simulation variables like time steps into this config block that is going from and and all along the way uh confirming that the the simulation is still running. So that was the reverse engineering of RX and Fur. So it's kind of like the original notebook was like a handmade chair and this was like reverse engineering it so that we could have this cartridge which gets run like a disassembled IKEA factory separating out the methods. Whereas the original example is great and really informative in terms of a blog post. Readable introduces things sequentially and shows what RX infer is doing and gets to those exact simulations. This shows like one path through the flow and it's a custommade artifact. And again that reverse engineering process was taking it into these different uh disassembled components but exactly reproducing it and then pulling out all the configuration into a config. Uh so that's the reverse engineering of the RX and first side. Now in the forward rendering this is going from a GNN file to that config because if we can if we know from that reverse engineering phase that getting to that config is going to allow the execution of the simulation. Then the other side of that meeting in the middle is getting the GNN file into that config format. So here is in in this examples folder a GNN style markdown file with all of the parameters that need to be specified in the config so that when you're in SRC and you run main is going to run all of the all of the the steps which are turned on of the 14 steps which are each their own numbered script with the same name module. Whichever ones you have enabled which is in the new file in pipeline module config. You can turn whichever of these modules on or off. That's different from last time. And uh that RX infer GNN example gets passed through step one checks the files. Step two does setups of the virtual environment. Step three does any tests. Step four, type checker output. We can look in the type checking and see for example that RX infer even though these resource estimates are are not currently specifically accurate especially compared

across different packages but the uh resources are calculated for that RX infer example just by virtue of it being a GNN. Then there's let's continue looking through the step five exports into a bunch of different outputs. Six makes visualizations and then of interest here is nine which renders what we want into the toml and 11 it's going to call the LLM. So here's the exports. This is the reexport of the GNN file and the export of the GNN file in other formats like XML and other graph formats. That's step five. Step six, visualization. we get the matrix visualization of all of the state spaces and ontology assertions of this model. So like this is an interesting example in this engineering RX infer notation name space A is often used for state transitions whereas in the active inference textbook B is used for state transitions but it's an example of why we don't want to have a welding between letters and ontology terms. we want to have the flexibility to use the same letters for different ontologies and language in in different settings. So it's perfect example of that of just the ontology um assertion space and all the variables can be described just because it was GNN file. Okay, now we get to step nine. Oh, it didn't run completely. To run it again. Python 3 main. Um, this will run the in the render in the render folder in SRC. In the RX infer section, there's a toml generator. So, this will read in the GNN and then output the toml which is the one that we knew that we wanted from the RX infer side. Then this script in the RX infer folders doc demonstrates the GNN RX infer pipeline for multi-agent trajectory planning. It performs two-step validation. So first it runs a baseline execution from the clone of the RX infer examples.jl which doesn't use GNN. those files for the example are all copied over into this uh folder right here multi-agent trajectory planning and then it's logged in the running of the script that the config was replaced with the one regenerated from the GNN. So it's a clean um functional replacement experiment to show that the GNN structured file here can be used as part of this bigger pipeline which includes the type checking, the category theory, all these these different kinds of ontology connections and yield the target TOML that we know will make the simulation work for the reverse engineered disarticulated RX example. So I hope that that has been conveyed. It took a long time, I think, overall to get here, but it had long been the triple play vision that we would have a unified plain text format for the math diagrams and visualizations and executable code. So, it's like, okay, how do we get triple play integrity? So we could have things like okay well you change the visual representation with a drag and drop and then it changes the executable code. It's like how is that going to work formally? Well, a lot of the examples scripts and scientific papers use like a all-in-one script which can make reproducing the example simpler and more monolithic but then give fewer degrees of freedom because you might be just like 700 lines in

the middle of the script and it's not clear how you change certain aspects more structural about the um model. So, how do we get to the ability for there to be a triple play where changes in one can be traced to changes in the other? And that was where Yakob and I were thinking maybe we have some sort of intra or infra lingua with the generative uh a generalized notation notation. Let's find that quote. Um, so RX Infer or any other software, we could try doing a reverse engineering of a PIMDP script gives the ability to set everything needed except the state space of the generative model. So there's still a lot to be explored in terms of how do we customize control flows and more simulation environment scale factors. Um and also uh uh a lot more to say. Um but suffice to say that for simple reproducing of standalone examples, this reverse engineering sort of forensic phase is a very important proof of concept. So in active inference, research models are often conveyed through assemblages of natural language, pseudo code, programming language, analytical formulas, and pictorial representations. In this paper, we present GNN as a flexible and expressive language tailored for expressing active inference models and encompass various relevant aspects of languages including ontology, morphology, grammar and pragmatics. By leveraging GNN as an active inferral lingua or interlingua, infralingua, superlingua, intralingua, we aim to bridge and respect gaps among different modeling approaches in order to facilitate interdisciplinary research. So how can this be used? Well, for our education, we can have these welltempered generative model infrastructural pipelines and specific, well documented, interpretable configurations that help us understand and communicate and figure out what different generative models and process flow elements are There's also a lot of engineering uses. So there could and will be many use cases where GNN is not a front feature of the product. It's just a useful, increasingly useful open source and stack for connecting the dots between existing examples and ways to modify and design with those motifs. And then increasingly forward use of this kind of metaprogramming program synthesis type approach to generative modeling which is going to have all these benefits for making them easier to design like just being able to prompt an LLM and say make me a GNN file that represents this like we'll do probably later in this stream. um all the way on through having the resource estimation. All these features that we want for any model can be abstracted and brought into this infrastructure grade pipeline. So that once there are more and more paths and understanding about how do you get more and more generically or in all these different situations to executable simulations. um using this pipeline and putting in the cartridge helps separate and and give a lot of reproducibility and expressivity to the compute that happens in between. So there's the reverse engineering approach which is just

taking scripts exactly as they are and within that language or in more of a port or transfer doing an exact reproduction effort and then from there maybe certain pieces can be flipped out but it's always good to know that we can exactly reproduce using the same variable names the same ontology assertions. So that becomes kind of like a substrate of well paper one modeled attention this way and in natural language they described attention this way. Paper two in a different programming language in a different natural language said this about attention. And here's how we can use ontology across the natural language and the computational elements to talk about where those are similar or different. Buzzing can be used for for different functions. So for example, once we have the config and the ability to separate out these different variables, we could see like what are the perturbations that are allowable because we didn't change any of the logic or the control flow. So for example, if the number of agents were simply changed to 50, then as it is currently written, there might need to be some other refactorizations of state spaces, which just changing NR agents wouldn't exactly do. It might just be part of a loop and then the loop would break because it's like, wait, there were only four because we only had four initial locations. So then you go okay okay okay how could we develop configuration spaces where number of agents at a programmatic level had a relationship to instantiations of matrices and other features like knowing that you needed to have as many of these blocks as you do agents. So in this exact situation again it was first just an exact reverse engineering but from that reverse engineering through insight and through program uh synthesis and fuzzing approaches we can explore like what happens when the DT is 0.01 or do a program sweep across generate simulations where DT ranges from this to this while the number of iterations ranges from that to that. So then that gives this ability to explore what are the functional and the performance aspects optimization and then we can go even further especially as the visualization and the disco pi features improve. So here not every single variable was captured by the disco pi category theory description but it wasn't the focus here but that would be something that could be worked on and again in general the goal is a wide range of defined and checked GNN formats ranging from discrete time continuous time all these different kinds of scenarios we continue to build out that space uh and neighborhood hoods of different models which can be interpreted a certain way. So there's a lot of things that could happen with the processing pipeline like how do we know that this PIMDP one we only want to render into PIMDP and the RX inver so okay should we include that information in the file and/or in the file name how should that be handled by the different modules those are the kinds of software improvements that would then allow us to when we

run it right now let's just say we had it perfectly working for rendering PIMDP to PIMDP and RX infer to RX infer we still might get 50% errors and then it would try to chase around fixing but it didn't need to be fixed. But then if we had some examples which are actually quite few if any where we have the same exact analytical situation being cross rendered into PMDP and RX infer. So those would be some interesting ways to check that all of the cross rendering is working well. Okay, returning to the agenda and if anyone has any uh suggestions or questions, write them. Otherwise, I'm just going to try to get through the rest of the agenda, go to some of the manual writing, see if there's some interesting thing that we could look maybe look at the documentation and just kind of have this um informal bring it together of all these recent developments over the last couple of days so that at least it's been worked through and put out there for those who want to explore it. and then it will return in in more forms over the coming months. So the golden spike was meeting in the middle between reverse engineering of the $n=1$ rx infer example into the components into the config form pipeline execution summary is a JSON file. So, what's cool about this is different steps can be tucked in with the arrow. So, you can look at just the logging for each step and it probably makes it more machine readable as well. But like here's the full logging for the step five and then step six visualization and so on. in terms of how the pipeline overall has changed. Big picture is still the exact same which is all the documentation is in docs. The RX infer script is in there for now just to be close to RX infer cloned repo and all the source code is in src. So you just delete the output folder and then run main.py here. It runs whichever of the modules are configured to run in config.py, it will run those sequentially. Each of them have a function and this is all very uh documented up in the repo. Okay. GNN computational science, deployment science, multi-agent systems, being able to design and describe these systems. And it starts quietly being able to write and read and modify GNN files in markdown that start to flow out through more and different places better and better. And that's just one trace through an information supply chain which hops across different operating systems and mines and servers and all of that. However, with this GNN entry point or meeting point, there's the ability to do some verified computing and tracing a little bit better from which generative model and which data inputs and so on are run how, when to yield what outputs. And that flow from abstract concept to concrete execution is unified slashunifiable. um some of the most uh important ways that people could explore from here getting the GNN repo examples working on their own machine making GitHub issues or comments with ways to improve it. um seeing what kinds of scenarios and value propositions they'd like to do themselves. Uh in which case

just go for it and report back, measure back or to to partner or support the institute so that we can have these kinds of tools being developed very functionally here. Okay, here's a little bit of a interlude, manual, manually written interlude to contrast with the large amount of computerenerated material. Then we will return to the documentation which has more computer generation. But this next piece is going to be a few thoughts that are selected from a larger writing. Um just to give a little bit of variety here, get some feedback and um again also more to be shared. So here we are in 2022 paradol textbook figure 1.2 to summarizing the high road and the low road. So we want to have notation systems and deployment methods for generative models with expressive methods for and nested addressable spaces of. So this next series is going to be the spaces of our capacity to do that task within describe, design, render, simulate, analyze, fit, assess performance of described models. Meaning there's a larger set of models we can describe than we can render. We can describe recipes we cannot cook. And that's a good thing. So zooming out there are these concentric spaces of generative models which we have different capacities in a given moment around for example let's do in the GNN examples make another GNN file fast and comprehensive for a full selfdriving car. Ensure all state faces and ontology ontological considerations are accounted for with good GNN style. So this is going to be an example of a GNN file that we certainly cannot render into a programming language in an executable fashion yet, but with some more constrain generation methods and better type-checking in ways of looking at which families of models and control flows we can run. then it makes it just an empirical fact that we can we can describe systems that we can't even visualize. Uh we could say it's a graph with 100 trillion nodes and then there's no computer that you have available that can visualize that many. But that didn't stop you from just writing the math on the paper or just doing a visualization. And then questions like the performance of a given model for a given data set in a setting is like seven horses ahead of the cart from what is possible to describe. So that's sort of like the math to application distance to travel which is like consider this equation and this is the attention factor and this one's the regret factor and this one's the the shame modulator and it's like you can just write those math equations however you want. No consideration to to too much other than just ideally what each of them mean. But when it comes to saying well which model would be a better fit of what that's several steps ahead of actually doing the testing and as long as that's known then there's some interesting cybernetic loops where you're like describing multiple kinds of analytical formulas with an insight which could be right or wrong into which one of them might be performant in a different way. And sometimes your intuition might be right and

it's kind of like a compressibility and there might be some other uh irreducible jumps that just like have to be computed on through. So when we're talking about performance of a generative model like this one is good at video games like this and that that way or this one's driving or this one's drone or recommending or resource allocating whatever domain or function you're making a generative model for performance on one benchmark. So the high jump approach or on a suite of different measures kind of like a decathlon. It's a relational measurement, assessment, or characterization that's secondary to the primary phenotype or embodiment. So map is not the territory. Generative models are maps. So we're talking about ctography, compositional cognitive cgraphy on territories which might be themselves abstract or might be embodied projects in and around the active inference ecosystem. they have performance and fitness wants and needs and so on. So someone might think okay well if this if I can design this kind of algorithm with this kind of runtime requirements with this performance by this date with this amount of funding then this is a viable method for us to have a business. So all these different kinds of performance or this should be more efficient or more resilient against this kind of intervention than this kind of model. All those kinds of of assessments that's all happening within this larger space of generative models which could be specified. So of course like even the ones that we describe are only a subset of what is possible. So those generative models, the larger spaces, the known unknowns and the unknown unknowns, they include the baroque, absurd, tiny, useful, cute, funny, and interesting. So here is the GNN for the self-driving car. So it's 566 lines. It could be made to be compliant with different kinds of natural language standards and these state spaces can be type checked for their coherence and their completeness. So, and then especially with a sumo type program like implementation of pure active ontology like definitions and other kinds of sentence fragments. Then there's all these different kinds of static checking that could come into play and resource estimation all the early stages of the pipeline basically up until but everything before 10 even for this model that an LLM just blasted out and that's not even with it having an MCP server. So with the ability for all of these methods within each module to be MCP model context protocol methods then compliant generation of arbitrary generative models can be dropped into the railroad track which already is going to reliably reach the triple play. So we have the free energy principle from the high road and baze theorem statistics updating learning on the bottom meeting at active inference. So that's why the low road is implementation specific and the high road is not because with the high road we want to be able to use FE and related methods on biological organisms and arbitrary levels of analysis or

abstract systems. So we want to be able to describe systems we can't build. Whereas the low road needs to be able to build what it's describing. So that would be that sort of bottom up like a building must have continuous connection even if it's through like tension or something in order to be x ft high. It just it's not it's kind of a classic uh cranes and skyhooks high road low road. So now going a little bit further here consider that we have pair wise information and distance measures today in terms of software 1.0 in terms of syntactic edit distance. So we can look at two different programs or inputs inputs as programs programs as inputs and look at their syntactic differences. More recently, we have gained accessibility of information and distance measures in terms of software 2.0. So AI, neural networks, machine learning as software as per kind of the Andre Karpathy post207 era, which is to say that we have these compressive distances, semantic distances, distances in so-called semantic information geometry model spaces. That's how RAG, it's how cursor works, how LLMs work. Here's what's not out there. Software 3.0 or it doesn't even matter or need a number, but cognitive computing distances. Now, that requires a kind of perspectival modeling which is least needed in software 1.0 0 where there's discrete numbers of changes to strings least to not needed to have multi-perspectivalism. Software 2.0 the perspectivalism is very inshed with the models geometry intrinsically. So we could look at the embeddings of a given input into multiple LLMs and consider each of the LLM's embedding in terms of compressive or semantic distances. So it's it's not that it's impossible to get multi-perspectivalism. It's just not it's being implicitly welded together in the trained state space of the LLM. What about having a sort of first principles active inference ontologically specified pre always already separation that gives true perspective cognitive semantic information differences differences that make differences for sophisticated cognitive agents. What's the accounting system domain specific notation and then general notation notation that lets us cover diverse and unknown cognitive phenomena addressing disparate systems from a first principles perspective which is the high road things we can describe and imagine that we can't necessarily build and then by specification that's larger than what we can build but where where the crane and the skyhook is when you have the active inference model that is at the triple play point where the math can be traced into the math. The variables data supply chain can be traced out to the next level and then the ontologies and the assertions that make up the rhetoric of the scientific model can be also separately broken out. And that's like pre- reverse engineered to be massively composable in this super useful way. So what does it look like for those phenomena here in doc cognitive phenomena. So here the read me in cognitive phenomena is going to link to each of these

folders some of which have files in them some don't. But just to show let's do drag them all in comprehensively go through this folder of phenomena. Ensure every subfolder has a technical accurate readme and a compliant GNN showcasing the phenomena. So here we have all these different subfolders of cognitive phenomena. This is the whole iguana. This is the unifying approach to different cognitive phenomena is notationally. It doesn't mean that there's functional unification per se. It's not that there is a relevant edge. First off, without a system of interest or scope in mind, it doesn't even really make sense to say, well, is attention related to language processing? Like yeah in principle you could design a system where there's any kind of relationship within any kind of anything because we're just imagining recipes. Then if you specify for a given system like for a red harvester ant nestmate does this relate to that? Well, then it starts to become an inquiry that can actually be pursued and whether or not they play some sort of synergistic or trading off role in a given situation as you've defined it, as you've set up the situation, etc., etc., etc. At least there can be a unified notation system so that within certain spaces those notation systems could be like fused, concatenated, tested for difference and so on. So COD 4.0 else on it doing incredible work just iterating comprehensively through and writing out uh the files as requested and then again to the point of like these are writing out recipes that are fantastical but then there's a way just like there's a way to build different software into Linux there's a to build approaches that bring you closer and closer to the Grand Central Station rendering assembly. So let's let it go through those examples read a little more. So for this part was to say for real um first principles design of cognitive computing with all of the relationality and the multi-perspectivalism that entails. We want to be able to do semantic information distancing in arbitrary multi-agent assemblies. And I'm contending that some kind of plain text format like GNN and associated packaging and so on can help support that coming to be the case even today with the kinds of things we can do. So in that light, the active inference institute supports reliable and effective applications of cognitive computing and active inference across domains. Different projects have different domains and systems of interest, but those are at the domain level and they're connected through the ontology, through the different tools. There's all these different places where somebody can focus on just a sub sub domain or anything. But how how how to gain access to a common set of tools and and and pipelines and methods? We can say now make a GNN for this domain because the notation and the rendering is being strongly typed. So we support that. reliable effective application by providing key services such as software. So that includes inference libraries, ones that we develop, ones that we just rebroadcast the development of of others, all kinds of

examples of ways that people are carrying out that terminal inference. And then we host things like the ontology generalized notation notation like what this stream is about cerebrum the case specific rendering of different generative models. There's sections in the docs about cerebrum and also pushing it further back education because the metal rails of the physical supply chain have their own supply chain leading to the education and the mind of the engineer. So education and outreach and awareness is also a part of it in terms of how do we make something that is lasting across generations and translating into ways of working together. Maintaining resources like tech trees, professional training, symposia, internships, inerson educational programs and so on. These are all different offerings and with hopefully partner organizations we can offer these better and different and other and more now just like dirt and asphalt roads low roads so real generative models that you actually build even if it's just in drafting like a real recipe you imagined let alone a real recipe that you made let alone one that you made like a business idea around. But real low roads are built basically only where and as and how needed less there be a bridge to nowhere. You could. It's like look being at the um Home Depot supply store and looking at all this material and thinking you could do this. It's like yes, you could. And if it were a quirky hobby or just a inexpressable curiosity, it it wouldn't even necessarily be a bad thing. It just it takes resources, not the least of which time. So considered in that way there's already many applications and implementations of variational approaches to cybernetics all but active inference all but the citation network even narrowly considered. So whether we look at our curated implementations and the dozens of explicit active inference implementations that we've curated in active blockfronts or whether we're just thinking well if it's a variational autoencoder and it's using free energy minimization to look at joint distributions of perception cognition action so on is that not basically doing free energy minimization on the particular partition and then we could even take that further we could think about this recent paper and recent guest stream of Tuya isomer on this triple equivalence and about the relationship between neural networks with their loss function B graphs the kind that we're working with more closely with the factor graphs and B graphs and RX infer and continuous turning machines how those have a triple equivalence so that that parenthetical was to Okay, however, however deep you're going to say active inference has already been applied, whether you think uh take a very narrow scalpel and say it's only been applied in these opensource papers that use exactly these criteria and these software packages and these calculations. Or if you zoom out and go actually every calculation cannot but be applying active inference. Wherever you are in that sort of frame,

there are some low roads and it is all dots within top-down notationally expressive analytically described spaces of generative models including uh second layer notational systems which we don't have or use here for operational grammar for kind of control flow. But just separating out this model probability distribution description question from the more operational side of of how it is used. for example, the neural network considered as a programming object and math object and then how it's used with what is sent to it when and all those different kinds of things. So there are some low roads maybe we think that the low road should be there should be more in this place or fewer in that place or the ones that are dirt should stay dirt and the ones that are this should be that. We we may have some preferences or some opinions on low road distributions and it's a separate related and separate topic to the expressivity of the high road and the generalizations which are not applying to specific cases. So consider this cognitive research tech infrastructural situation analogous to the metallurgy and alchemy of high precision calipers which can be used to describe and measure machine parts during a material industrial revolution. Hence our focus on accessibility, rigor, applicability, education, sovereignty, supply chains and security. To recap that manual section, common dialectic rhetoric in active inference related to the low road and the high road. We want to be able to understand where are we talking about recipes that we can and can't do different things with. the the sort of in that uh cooking lens. The best GNN package would be for very large to all known generative model types. What is express what is expressed slashexpressable through natural compositional language could be taken on through at least s some steps like the type checking the category theory um the ontology even if it wasn't known how to compute certain things still the description could be analyzed on its own So in terms of applying active inference and that being used in high reliability settings, even if it's just high reliability and important and meaningful for you, we're talking about performance, trade-offs, fitness, phenotype, adaptability in specific situations. which is very far down the road from imagining the recipe. So it's one thing to be like I wonder how a nested generative model for this and that would work for this and if it could be used for this and that important function and then dot dot dot dot dot dot dot dot dot dot here's my business plan that I believe is going to work for how we're going to do this and that. So that there's a big space between and that's where the information supply chain for generative models comes to be and whether that's going to be something that has to be blazed through by individuals and the sorts of failures and trade-offs that would incur and/or which parts of that coast to coast are their well characterized methods for. So in order for people who care about performance to be able to even get to the point of

assessing it and not going down this lifelong learning to write generative models specialty as it has largely been at this point. In order for it to break out of that, having infrastructure that takes on this scope is going to be critical. It will also provide some deeper directions into the organization of education and research and professionalism through connection with information sciences cognitive computing. So meanwhile, let's update GitHub. Meanwhile, the LLM was going through the cognitive phenomena folder and writing files. So here's the learning and adaptation And this sets up a a multi-channel discussion. On one hand, there's the low road discussion. Do we really think that there should be this many state spaces? No, I think there should be 25 dimensions or let's have this as a variable and then do a sweep across how many dimensions or what what are we really talking about with the bistable perception? Are we talking about the ballerina spinning or are we talking about the duck rabbit or that? So it supports the technical discussion of state spaces or for for a bio regional setting. What variables are we taking into account in this model? How? And then there's there's this sort of duel which is just separate from the state space is like what are we doing here? Should we be modeling this? What are the feedbacks of us modeling this in general and in specific ways? So just very interesting, useful moments that can have some describable steps that take them from basically uh through the steps of cognitive systems. design. There's a few other new folders documentation. [Music] Um, some are related to other packages. There's also the tutorials. Let's look at this in GitHub. I'll look at these then maybe do one or something random. So if anyone has a comment or a question or wants to see like a GNN model or some function then just write it in the chat some tutorials. So first making a simple GNN model basic perception a simple model with one hidden state and one observation. So this is like the first model in step by step hidden state with two categories observation with two categories connections parameterization. Here's how you run main to go on specific steps to check at how it works and then more future tutorials. pipeline has some more information on the pipeline itself. pipeline architecture has more on the control flow, the purposes of the different modules. Um probably the main control flow could be redesigned or refactored but it works as it is to uh call these functions which call into folders. So hopefully that gives some touch points. That's the forward engineering. Let's let's separate out one RX example scripts folder. Let's do hidden markoff model and then if it is too challenging maybe try the coin toss. Okay, that was just the cognitive phenomena just showing you can make subfolders with lists and so on and it will just rifle through them. Okay, refactor this example unless anyone makes a specific comment or question. Start by just running the script plainly. Sometimes it it works in the

notebook format. uh or other times it'll hit a Julia error. Okay, there's the error. I'm just this I'm just gonna tell cursor this was from that notebook so that it runs as a script. Here is the error we get when we run it. Now, this could be done to first to to to more cleanly do it. It probably could be focus on having this only be one script and f and get a single script version like I did with a multi- agent or in this case let's just go separate it into as many files in the folder as you want. Okay. So, it's going to start to get this hidden markov model RX infer program running. And meanwhile, this this could be done after fully disassembled, but let's just try it as per. And then I'll just drag in the existing GNN files. write and make to this folder a GNN file or the RX infer hidden mark model example all the parameterization it would need as a GNN And then where it's going to go is in the app model block. Instead of saying B is this hard-coded matrix or A is an identity matrix with a 33 those variables will be extracted into this TOML or maybe even we can go right to GNN that's the reverse engineering then the forward engineering is in render RX infer folder say I'm trying to first off if this could already be a GNN file we don't even need to do the toml meet in the middle but if the parser can just directly use information then it's good to go okay Now this config this GNN for the RX infer POM DP will not work in the multi- agent trajectory avoidance example and vice versa. So that is just to say there's a lot more that we can diagnose and refine between which GNN can be taken to what step in what uh language. But this hmm like for example we could let's just say all changes to just ask mode. So it just it's going to respond to not make code edits. How are these files similar and different? Greetings Matias Conrath. So imagine large libraries of GNN files, some forensically reverse engineered from examples and papers, others generated from program synthesis. And we could do different kinds of probabilistic and deterministic analyses on oh this this one differs all but for this or 80% of this had that but this one didn't or something like that. Key differences similarities as mentioned earlier like in the PMDP A is the observation matrix. But in the RX infer HMM, A is the state transition matrix. Okay, let's see how the refactoring is going. So here is that splitting from the uh single craft artifact, the local organic handmade farmers market style into the IKEA industrialized high reliability information supply chain format. Maybe it's hanging, but it is at least getting broken up into separate parts. Just letting it know that it's hanging. Okay. Triple play. Okay. while it's fixing the HMM. Various strategies and tactics may be helpful when employing GNN expressions in different settings and using a spectrum of model precisions, eg from informal conversation to a beautiful presentation to a fully documented reproducible research product. Model precision should be balanced against the need for comprehensiv comprehensivity comprehensibility

could have been comprehensivity too and ease of communication. Broadly strategic considerations for expressing GNN include the overall approach to communication such as the choice of modality and level of technical rigor applied for a given audience. Tactical include specific techniques for enhancing clarity and understanding such as using visual aids, examples, and analogies. AI art using models for inference. Data on inference models as data markdown texts as data as GNN files meeting the visualization plain text markdown files. meeting RX infer with additional visualization and methods. Kind of the magic of seeing GNN work. Here we go. So here on the right side at least something is happening here it might be one moment too early to do it, but to now add to the pipeline add to the RX infer render pipeline. If if we can reverse engineer the HMM to to use the pure markdown, we don't even need to render it to this intermediate TOML. But part of the reason why I wanted to go to the TOML was I did that standalone in another repo just to get to a a controlled input to show that there was a middle to be met at. Now that we know that there is a middle to meet at, we can basically just go straight to GNN and have example specific or category specific parsers, but that still needs to be rendered in the GNN uh folder because the other the side information like the utility functions that this HMM model needs this wouldn't be in a GNN file. So we need these kind of kind of like a car body and then the GNN parameterized parts are like the engine but there's a lot of functions in code and so the way that was being done in the multi- agent setting was just by copying the folder minus the config if here it made the HMM GNN in the GNN folder. So from the point of view of the HMM script folder, it still does not have a GNN file. But if this if it will succeed on run, then we will have succeeded in getting to this face. So this one we skipped going to a single functional script went straight from the notebook uh extract to the modular software. So one outcome of this step is a specific config file. Another emerging approach is directly to target read write into GNN format directly. Still needs the rendering extra parametric information. training program environment and flow information that's not part of the generative model. All right, here we go. So, we got the now it's working for the hmm inference. So, I'm going to delete the original one or it could I'll move it to archive or maybe it wants that one. Okay, ceue up the next question. Learn again to confirm function comprehensively output more in standalone from standalone file visualizations of free energy values. is so several prompts depending on the example to disassemble it. Meanwhile, let's try let's try another approach which is not to disassemble it. So this will be more um first you have to get it running as a script or we could develop a plugin that goes directly to the notebooks but it's so much nicer to have it as these which can be concatenated

into a notebook too but at least you know it's going to work. So while that HMM is happening let's do another approach which would be for this example write a GNN file like for comprehensively setting all the needed information for the bike rental demand example and write a Julia parser that confirms that the GNN you wrote perfectly align with the initial example. Okay, let's see how this goes. Remove any unneeded files too. Archive. Confirm that when running the main script, all data and visualizations are saved to a timestamp folder. Okay. So this is this would be an example of if we can skip even further steps by directly inserting in the GNN repo the state space that needs to be created for an RX infer example to run. Okay, let's see what Julia. Okay. So running comprehensive HMM if anyone has any last uh ideas or questions otherwise I'm going to take it just a little bit further with this HMM and uh bike rental then maybe one triple play who's on first and seems to be hanging There it says and improve and rerun to confirm. Here's the validation parser test. This one also seems to be hanging because It's a small. Okay. All right, let's just start generating some of the baseball stuff. Meanwhile, Um, next steps, seeing who watches and comments and emails and makes issues and contributions on GitHub. I'm going to keep working on the package and the flow and the and putting uh hopefully some useful and interesting work out but very much open to people's support or or feedback on that. That pretty much is what I wanted to cover today. Okay. One baseball related analysis and then that be Golden who's on first. Oh, that was interesting. Only one of us can be the first to do something. Wow. Only in the tone. Well, there's only in the tone of the golden spike. I'm going to be the first to say that I'm not sure what the golden spike is. explore greatly within that topic in allegorical sequence involving show not tell approaches to category theory recursive selfdevelopment styled exoterically as pun laden quote who's on burst style 1920s illusion environment make the dialogue Totally convey in plain quote basketball down the middle style generalized notation notation GNN Interesting little error there, huh? Can't be deleted. Very strange. Gone. all the technical elements, all the main SRC elements, all the documented connections. Write this all out vastly and creatively. All right. If anyone has any last question, do write it. Otherwise, look forward to some collaboration on the tooling and some fun interesting discussions going a little more broadly. How does this relate to to how we learn and apply active inference? All these topics Yeah, it will be uh interesting to see what prompt sequences, what measures of are useful for these kind of nested documentation and code folders. How do you know when to stop? All right, here it is. Let's look at it in GitHub. Okay, I'll just read it. I'll read it and then if anyone writes any comment I might read that and then that will be the end. All right. The golden spike

dialogue. Who's on first base in generalized notation notation scene. 1920s railroad depot. Two gentlemen in bowler hats stand before a massive scheduling board covered in cryptic notations. One holds a golden spike. The other clutches a stack of papers marked GNN pipeline specification. Abbott. Well, now Castello, we've got ourselves quite the categorical conundrum here at this generalized notation notation depot. Say, Abbott, what's all this gobbledegook about spikes and rails and `s_f031` type int? Looks like someone's been playing alphabet soup with mathematics. Ah, my dear fellow, you've stumbled upon the very essence of our enterprise. that there's a state space variable, the foundation of our active inference railroad system. Active what now? I thought we were building a railroad, not running a gymnasium. No, no, no. Active inference, my good man. It's about how minds make models of the world. Think of it like this. Every train car needs to know where it's going, where it's been, and what's around the bend. So, the train cars are thinking precisely. Now, let me show you our 14 step construction process. First, we start with step one. The GNN parser. The what parser? The GNN parser. It reads our standardized notation files. Oh, I see. So, GNN stands for good night Nancy. Generalized notation notation. That's what I said. Good night, Nancy notation. No. No. GNN. It's our textbased language for specifying these thinking train systems. Well, why didn't you say so? So, this Nancy notation tells the trains how to think. Let's move on. Step two is setup. Absolutely critical, mind you. If setup fails, the whole pipeline stops dead in its tracks. Like a locomotive with a busted boiler. Exactly. Then step three runs our tests. Tests. What kinds of tests do the train cars take written exams? In a manner of speaking? Yes. We validate their state spaces, check their connections, ensure their probability matrices are properly stoastic. They're what? Astic matrices stochastic. It means the probabilities sum to one. Like making sure all the passengers on a train car add up to the total capacity. Ah, so if I got three passengers in car A and two in car B, I better have five total passengers. Now you're getting it. Step four is our type checker. It estimates computational resources. Type checker. Is that like making sure the passenger manifest has everyone's name spelled right? More like making sure we have enough coal for the engine and enough track for the journey. It looks at your model and says, "This will need X amount of memory, Y amount of processing power." Smart. What's step five? Export. We translate our GNN models into different formats. JSON, XML, GraphML. Sounds like a whole lot of MLS to me. What's GraphML stand for? Grumpy Manatees Language. Graph markup language. Oh, that makes more sense. And step six, visualization. We create beautiful diagrams showing how all the state factors connect to observation modalities. Hold up there, fancy pants. State factors.

Observation what allies? Think of it this way. State factors are like different aspects of what's happening inside the train car. Maybe factor zero is how hungry the passengers are and factor one is how tired they are. Got it. An observation. What's its modalities? Those are the different ways you can sense what's going on. Maybe modality zero is what you hear. Modality one is what you see. So the hungry, tired passengers in the train car can be heard and seen. Bingo. Now step seven gets really interesting. That's our model context protocol integration. Model context what a call protocol. It's like having a telegraph system that lets different railroad companies talk to each other using the same language. Ah, so like Western Union but for thinking trains. Exactly. Step 8 handles our ontology processing or whatology. Ontology. It's like having a master dictionary that defines what every term means in the active inference world. So if I say belief and you say belief, we both mean the same thing precisely. Now, here's where it gets really exciting. Step 9 renders our models into executable code for different simulation environments. Executable code? Are we hanging somebody? No. Executable means runnable. We translate our GNN specifications into Python code for PIMDP or Julia code for RX infer. Hold the phone there, Abbott. PIMDP, RX infer. These sound like patent medicines. PIMDP is Python for Markov decision processes discrete active inference simulations. RX infers reactive message passing in Julia. So PIMDP is like Python flavored train scheduling in a sense. Yes. And RX infer is like having a telegraph network where messages bounce around updating beliefs in real time. Telegraph network. I like that. What's step 10? Execution. We actually run the code we just generated like firing up the locomotive. Exactly. Step 11 is where we bring in the big guns. LM integration. LM. Let me guess. Large language models. He knows that one. Very good. We use AI to analyze and enhance our GNN models. Provide natural language explanation. So the thinking trains get help from other thinking trains. It's turtles all the way down, my friend. Step 12 is where things get mathematically sublime. Disco pie translation. Disco pie. Are we having a dance party with dessert? Disco pie. Distributional compositional Python. It's category theory for string diagrams. Category theory. What's that got to do with railroad categories? Fright, passenger, mail car. Now we're getting to the philosophical heart of it all. Category theory is mathematics that studies how things compose together. Like how you can connect train cars to make longer trains. So if I got a dining car and I connect it to a sleeping car, I get a dining sleeping car. More like you get a composed system where the properties of both cars work together. The beauty is in the connections, the morphisms between objects. Morph whatisms? The arrows. the relationships, how one thing transforms into another. In our railroad, it's how beliefs flow

between different parts of the system. Like how the conductor's belief about the schedule flows to the engineer's belief about when to blow the whistle. Brilliant. You've grasped the you've grasped the essence of categorical composition. Step 13 uses Jax for high performance evaluation. Jax, is that short for Jackson? just another executable. It's Google's library for automatic differentiation and just in time compilation. Automatic different what? It's like having a mathematical microscope that can instantly tell you how changing one little thing affects everything else in your model. Like if I change the coal input, it automatically figures out how that changes the train speed, passenger comfort, arrival time. Exactly. And just in time compilation means it optimizes the code right when you need it. Like having a master mechanic tune your engine on the fly. Slick. Step 14. Site generation. We generate beautiful documentation websites showing off all our work. Like a fancy brochure for the railroad company. Precisely. Now let me show you the recursive beauty of this whole system. Recursive? That sounds like something you need a doctor for. No. No. Recursive means self-referential. The system can model itself modeling things. So the thinking train can think about itself thinking about thinking. Now you're getting into the philosophical deep water. Each GNN model is like a mirror that can reflect other mirrors creating infinite depths of self-awareness. Like when you stand between two mirrors in a barber shop and see yourself going on forever. Beautiful analogy. And that's where our state space blocks come in. State space blocks are we're building with Legos. Now think of them as the blueprint sections. Each block defines variables with their dimensions and types like `s_f31 type int` means state factor 0 is a 3x1 integer array. So it's like saying car number one holds exactly three passengers and we count them with whole numbers precisely. And our connections show how these blocks relate. We use greater sign for directed relationships hyphen for undirected ones. Like engineer to conductor means the engineer gives orders to the conductor. Exactly. And passenger hyphen passenger might mean passengers can talk to each other both ways. # cerebrum. This is starting to make sense. What about those matrix things he mentions? Ah, the ABC and D matrices. The heart of active inference. ABC D. Sounds like we're back in kindergarten. Matrix A is your observation model. It tells you the probability of seeing something given what's actually happening. Like the probability I'll hear the whistle given that the train is approaching. Perfect. Matrix B handles transitions. How things change over time, possibly based on actions. Like how pulling the brake lever changes the train speed. Right? Matrix C represents preferences. what outcomes you'd prefer to see, like preferring to arrive on time rather than late. And matrix D gives you your priors, what you believed before you had any

evidence, like believing the train is probably on time when you first arrive at the station. You've mastered the fundamentals. Now, here's where the golden spike comes in. Finally, I was wondering when we'd get to the spike. Is it made of real gold? The golden spike represents the moment when all these separate components connect into one unified system. When PIMDP talks to RX infer when disco pi diagrams execute as jacks code. When loom analysis enhances human understanding like when the eastern and western railroads finally met in Utah. Exactly that ceremonial spike joining two great endeavors into one transcontinental system. RGNN is the golden spike of computational active So this whole contraption, the 14 steps, the thinking train cars, the matrix algebras, the category theories, it all comes together to make one big what. A standardized way to describe, validate, visualize, and simulate how minds work. Whether it's a human brain, an AI system, or a whole ecosystem of interacting agents. Agents, are we talking about secret agents now? Computational agents, anything that acts based on beliefs and desires. Your train conductor is an agent. He believes certain things about the schedule and desires to keep everything running smoothly. And all these agents can be described using this GNN notation. from the simplest thermostat to the most complex social system. Each one gets its state states blocks, its connection diagrams, its ABCD matrices and then they can all talk to each other through this model context protocol telegraph system and be simulated in PIMDP or RX infer visualized with pretty diagrams analyzed by LLM translated to category theory optimized with jacks and documented on beautiful websites will all be hornswaggled. So this golden spike connects not just two railroads but 14 different ways of understanding thinking systems. 14 steps, infinite possibilities. From parsing text files to generating websites, from validating syntax to executing simulations, it's all connected by the golden thread of standardized notation. And anyone can learn this GNN language. It's designed to be human readable. Markdownbased files with clear sections. Model name, state space, block, connections, initial parameterization. Sounds like a recipe for building thinking trains. More like a recipe for understanding how thinking itself works. The recursive beauty is that we use thinking systems to design better thinking systems. Like using a train to design better trains. Exactly. And the category theory ensures that when we compose these models together, the whole system remains mathematically consistent. like making sure when you connect train cars, they actually stay connected and don't go flying off in different directions. Beautiful. You've grasped the essence of compositional modeling. Now, shall we drive that golden spike? You bet. But just one more question. When we drive the spike, does it start the whole 14step pipeline? Indeed, it does.

From the first GNN parse to the final website generation, it all begins with that ceremonial connection. Then let's do this thing for the transcontinental railroad of computational consciousness for active inference in the standardization of thinking systems everywhere. Three cheers for GNN. Hip hip hooray. Wait, Abbott. What happens after we drive this bike? Why then we start working on GNN version 2.0? Of course. Oh no, here we go again. Right. Another work in exactly like William Blake illuminated poem status woven densely with actual integrated William Blake. quotes and styles about implicitly and explicitly. Pretty funny baseball ones. Delete the output folder. src Python 3 main. Y regenerates the output folder. And then in another one run pipeline validation and check comprehensively for full functionality of and updated streamlined documentation. Still on step six with the visualization those there's the outputs exports. So, like that self-driving car that we made. That's probably why the visualization is taking a long time because now we have the self-driving car GNN. So, I'll pull. That's the trade-off with having a bunch of the GNN files actively there, but it'll be cool to see. Archive has a bunch of GNN. So, we can just run that one. while it's writing the Blake in. This is very effective having the agent run the pipeline and then test it itself. Now let's move to seven. So like this is the attempt to write the TOML falling back on default values trying to write the PIMDP script. So, not a solved uh problem yet, but okay. Let's see if this will work. Otherwise, it I'll just end it here. But I'll wait one minute, have a drink of water, see if anyone writes a comment. funny. Each of the three times it's tried to write it, it's called it a different file name. Here's the visualization of the self-driving car. Yeah, it's massive. But so cool. All the variables and ontology terms for an LLM generated output. Four-fold vision of GNN. I hope that one works. Okay, looks like it is not to be like it is to be unwritten. So, thank you for watching. Hope people can find out how to engage or support as they see fit. contact blanket activeinference.institute if you have any questions or anything whenever that may be. So till next time